Chapter 5 Class Design



Contents



- Identifying Appropriate Classes
- Determining Associations Between Classes
- Identifying Attributes
- Identifying Operations
- Library System Class Diagram
- Student Registry System Class Diagram

5.1 Identifying Appropriate Classes



Identifying Classes



- List all the nouns in the specification.
- Prune the list:
  - Convert plural nouns to their singular form.
  - Eliminate nouns that represent objects. Replace them with generic nouns. For example, use "client" instead of "John Smith."
  - Eliminate vague nouns.
  - Eliminate nouns that are class attributes.
- Group the synonyms and then choose the best name for the class from the group. For example, "user" and "client" are synonyms. In a bank system, the best name is "client" because the system may have two types of users: the clients and the bank's employees.
- Select the classes that are relevant to the system.

Identifying Classes



- Look for more relevant classes. Search the application domain for:
  - Physical things. For example, "person," "book," and "computer."
  - Roles played by persons or organizations. For example, "employer" and "supplier."
  - Objects that represents an occurrence or event. For example, "system crash," "flight," and "mouse click."
  - Objects that represent a relationship between other objects in the model. For example, "purchase" (related to "buyer," "seller," and "merchandise") and "marriage" (related to "man" and "woman").
  - People who carry out some function. For example, "student" and "clerk."
  - Places. For example, "library," "classroom," and "bank."
  - Collections of objects, people, resources, or facilities. For example, "catalog" and "group."
  - Concepts or ideas that are intangible. For example, "money" and "bank account."

Identifying Classes



- A client is a person that has one or more accounts.

Identifying Classes



- Client
- Person
- Account

5.2 Determining Associations Between Classes



Determining Associations Between Classes

- Create an *n x n* table where *n* is the number of classes. Label the rows and columns with the class names.

	Client	Person	Account
Client			
Person			
Account			

Determining Associations Between Classes



- To identify the inheritance (specialization/generalization) relationships, for each cell in the row *A* and column *B,* ask the following questions:
  - Is an instance of class *A* an instance of class *B*?
  - Is an instance of class *B* an instance of class *A*?
- If the answer to both questions is *yes*, then the class names might be synonyms.
- If the answer to the first question is *yes*, then class *A* is a subclass(specialization) of class *B*. Mark the cell in the row *A* and column *B* with an *S*.
- If the answer to the second question is *yes*, then class *A* is a superclass(generalization) of class *B*. Mark the cell in the row *A* and column *B* with a *G*.

Determining “is a”

	Client	Person	Account
Client		S	
Person	G		
Account			

Determining “has a”

- If there is no association between class *A* and class *B*, mark the cell with an *X*.
- If there are one or more associations between class *A* and class *B*, then insert the association attributes. For example, "pilot," "wife," "ownedAccounts," and "clients."

	Client	Person	Account
Client	X	S	ownAccount
Person	G	X	X
Account	X	X	X

5.3 Identifying Attributes



Identifying Attributes

- Attributes are the data that an object is responsible for knowing and maintaining. The steps for identifying the class attributes are the following:
  - Look for adjectives and possessive phrases such as *"the X of Y"* and *"Y's X"* in the system specification. For example, "number of the account" and "client's name."
  - Use your knowledge of the application domain to define the set of attributes needed for the system being developed.

Identifying Attributes



- Person：name
- Client：
- Account：numOfAccout，password，balance

5.4 Identifying Methods



Identifying Methods



- To identify behaviors, look for **verbs**. For example, the statement

  "the client deposits money into the account"

  indicates that class Account should define a method deposit.

- Use your knowledge of the application domain to define the set of methods needed for the system being developed. Look in the application domain for actions that:
  - Create and initialize new instances.
  - Set and get values of attributes.
  - Load to and save from persistent storage.
  - Destroy instances.
  - Perform calculations using an object's values.
  - Output or display a result.
- If there are any collections held by the object, include the methods needed to add, remove, and access elements of these collections.

Identifying Methods



- Person:
  - getName()
  - setName()
- Client:
  - getAccount()
  - setAccount()
  - addAccount()
  - deleteAccount()
  - findAccount()
- Account:
  - deposit()
  - withdraw()
  - getBalance()

5.5 Library System Class Diagram



Modeling the Library System



- Specification of the Library System
- Identifying Classes
- Identifying Relationships
- Identifying Attributes
- Identifying Methods
- Modeling Using UML

Specification of the Library System



- The library system tracks the items checked out by borrowers.
- The system contains a catalog of the items owned by the library. There are two kinds of catalog items: books and recordings. All catalog items are identified by a unique code. (If the library owns several copies of the same book or recording, each copy has a unique code.) The information for each item includes title, year, and availability. An item is available if it is not checked out.
- In addition:
  - The information for a book includes the author and number of pages.
  - The information for a recording includes the performer and format (*CD* or *tape*).
- The system contains a database of the borrowers. Each borrower has a unique identification code in addition to a name. The system maintains a list, for each borrower, of the catalog items checked out.

Specification of the Library System

- In the library system, the user should be able to:
  - Display the catalog by listing the code, title, and availability of each item.
  - Display a catalog item.
  - Display the borrowers by listing the identification code and name of each borrower.
  - Display the catalog items checked out by a borrower.
  - Check out a catalog item by adding the item to borrower's list of borrowed items.
  - Check in a catalog item by removing the item from borrower's list of borrowed items.

List the nouns in the system specification

- library system
- Book
- CD
- Items
- Recording
- Tape
- Borrowers
- Copy
- Database
- System
- Information
- Borrower
- Catalog
- Item
- identification code
- Library
- Title
- Name
- Kinds
- Year
- List
- Catalog Items
- Availability
- User
- Books
- Author
- (borrower) code
- audio items
- number of pages
- list of borrowed items
- (item) code
- Performer
- Copies
- format

Prune the list by eliminating the following nouns

`(item) code, title, year,availability`	These nouns are attributes of a catalog item.
`author, number of pages`	These nouns are attributes of a book.
`performer, format`	These nouns are attributes of an audio item.
`CD, tape`	These nouns are values for the attribute *format*.
`name, identification code, (borrower) code`	These nouns are attributes of a borrower.
`user`	This noun is an element outside the library system.
`information`	This noun is a generic term for the class attributes.
`kinds`	This noun is not relevant to the system.

Group the synonyms



`CatalogItem`	items, catalog items, copies, copy, item
`LibrarySystem`	library system, system, library
`Borrower`	borrowers, borrower
`Book`	books, book
`Recording`	recordings, recording
`BorrowerItems`	list, list of borrowed items

Select the classes that are relevant to the system



- LibrarySystem
- Catalog
- CatalogItem
- Book
- Recording
- Borrower
- BorrowedItems
- BorrowerDatabase

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem								
Catalog								
CatalogItem								
Book								
Recording								
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem								
Catalog								
CatalogItem				G				
Book								
Recording								
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem								
Catalog								
CatalogItem				G	G			
Book								
Recording								
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem								
Catalog								
CatalogItem				G	G			
Book			S					
Recording								
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem								
Catalog								
CatalogItem				G	G			
Book			S					
Recording			S					
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem		catalog						
Catalog								
CatalogItem				G	G			
Book			S					
Recording			S					
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem		catalog						borrowerDB
Catalog								
CatalogItem				G	G			
Book			S					
Recording			S					
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem		catalog						borrowerDB
Catalog			item					
CatalogItem				G	G			
Book			S					
Recording			S					
Borrower								
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem		catalog						borrowerDB
Catalog			item					
CatalogItem				G	G			
Book			S					
Recording			S					
Borrower							borrowerItem	
BorrowerItems								
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem		catalog						borrowerDB
Catalog			item					
CatalogItem				G	G			
Book			S					
Recording			S					
Borrower							borrowerItem	
BorrowerItems			items					
Borrower Database								

Identifying Relationships

	LibrarySystem	Catalog	CatalogItem	Book	Recording	Borrower	BorrowerItems	Borrower Database
LibrarySystem		catalog						borrowerDB
Catalog			item					
CatalogItem				G	G			
Book			S					
Recording			S					
Borrower							borrowerItem	
BorrowerItems			items					
Borrower Database						borrower		

Identifying Attributes

LibrarySystem	catalog, borrowerDB
Catalog	items
CatalogItem	code, title, year, available
Book	author, numberOfPages
Recording	performer, format
Borrower	name, id, borrowedItems
BorrowerItems	items
BorrowerDatabase	borrowers

Identifying Methods



LibrarySystem	displayCatalog() displayCatalogItem() displayBorrowerDatabase() displayBorrower() checkIn() checkOut()

Identifying Methods



Catalog	addItem(item:CatalogItem) getItem(index:int):CatalogItem getNumberOfItems():int getItem(code:String):CatalogItem

Identifying Methods



CatalogItem	getCode():String getTitle():String getYear():int isAvailable():boolean setAvailable(value:boolean)

Identifying Methods

Book	getAuthor():String getNumberOfPages():int
Recording	getPerformer():String getFormat():String
Borrower	getId():String getName():String getBorrowedItems():BorrowedItems

Identifying Methods

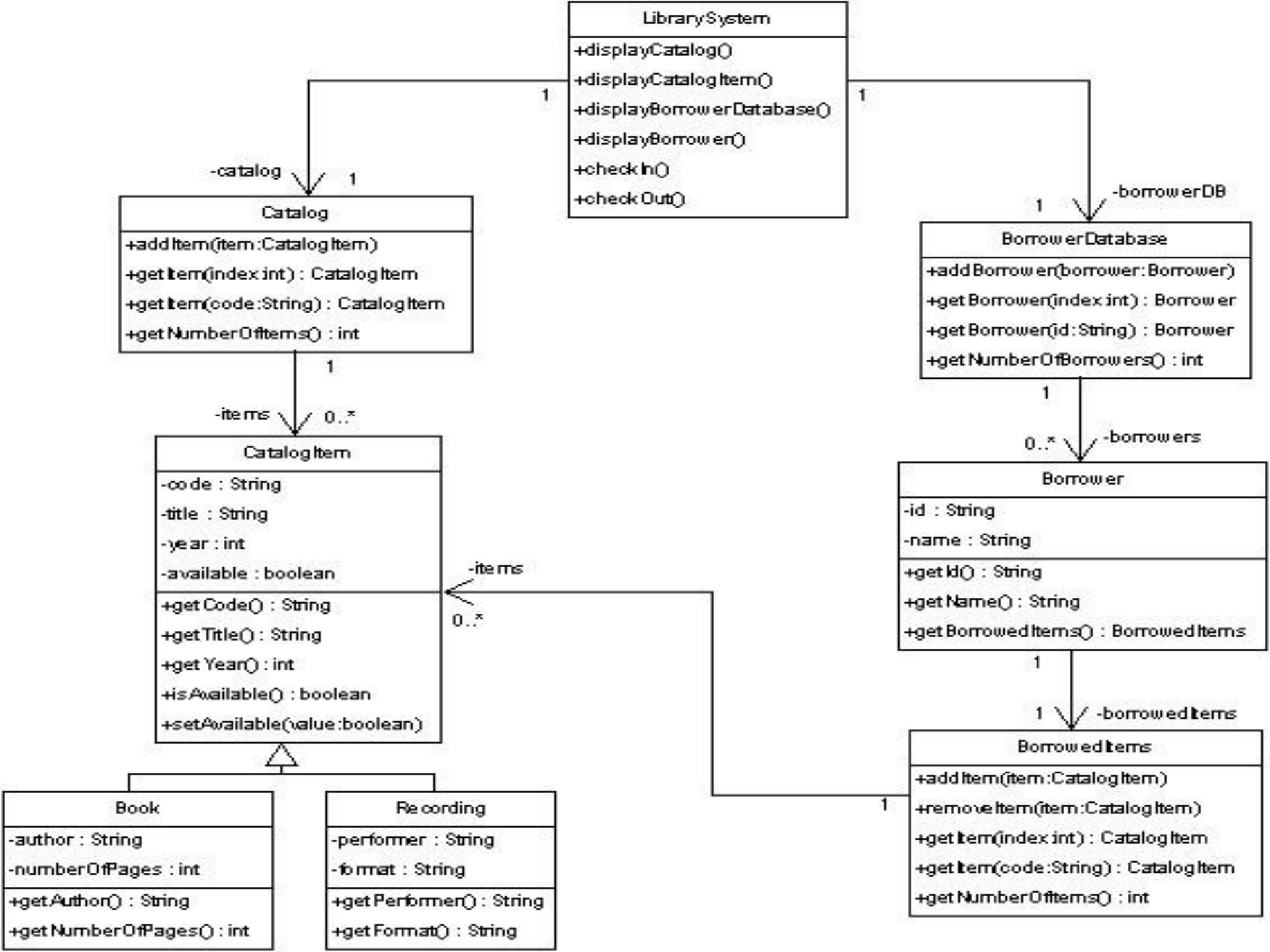


BorrowerItems	addItem(item:CatalogItem) removeItem(item:CatalogItem) getItem(index:int):CatalogItem getItem(code:String):CatalogItem getNumberOfItems():int

Identifying Methods



BorrowerDatabase	addBorrower(borrower:Borrower) getBorrower(index:int):Borrower getNumberOfBorrowers():int getBorrower(id:String):Borrower


LibrarySystem
+displayCatalog()
+displayCatalogItem()
+displayBorrowerDatabase()
+displayBorrower()
+checkIn()
+checkOut()
1
1
-catalog
1
Catalog
+addItem(item:CatalogItem)
+getItem(index:int) : CatalogItem
+getItem(code:String) : CatalogItem
+getNumberOfItems() : int
1
-items
0..*
CatalogItem
-code : String
-title : String
-year : int
-available : boolean
+getCode() : String
+getTitle() : String
+getYear() : int
+isAvailable() : boolean
+setAvailable(value:boolean)
Book
-author : String
-numberOfPages : int
+getAuthor() : String
+getNumberOfPages() : int
Recording
-performer : String
-format : String
+getPerformer() : String
+getFormat() : String
1
-borrowerDB
BorrowerDatabase
+addBorrower(borrower:Borrower)
+getBorrower(index:int) : Borrower
+getBorrower(id:String) : Borrower
+getNumberOfBorrowers() : int
1
0..*
-borrowers
Borrower
-id : String
-name : String
+getId() : String
+getName() : String
+getBorrowedItems() : BorrowedItems
1
1
-borrowedItems
BorrowedItems
+addItem(item:CatalogItem)
+removeItem(item:CatalogItem)
+getItem(index:int) : CatalogItem
+getItem(code:String) : CatalogItem
+getNumberOfItems() : int
-items
0..*
1

5.6 Student Registry System Class Diagram



Modeling the Student Registry System



- Specification of the Student Registry System
- Identifying Classes
- Identifying Relationships
- Identifying Attributes
- Identifying Methods
- Modeling Using UML

Specification of SRS



- We have been asked to develop an automated Student Registration System (SRS) for the university. This system will enable students to register online for courses each semester, as well as track their progress toward completion of their degree.
- When a student first enrolls at the university, he/she uses the SRS to set forth a plan of study as to which courses he/she plans on taking to satisfy a particular degree program, and chooses a faculty advisor. The SRS will verify whether or not the proposed plan of study satisfies the requirements of the degree that the student is seeking.

Specification of SRS



- Once a plan of study has been established, then, during the registration period preceding each semester, students are able to view the schedule of classes online, and choose whichever classes they wish to attend, indicating the preferred section (day of the week and time of day) if the class is offered by more than one professor. The SRS will verify whether or not the student has satisfied the necessary prerequisites for each requested course by referring to the student's online transcript of courses completed and grades received (the student may review his/her transcript online at any time).
- Assuming that (a) the prerequisites for the requested course(s) are satisfied, (b) the course(s) meet(s) one of the student's plan of study requirements, and (c) there is room available in each of the class(es), the student is enrolled in the class(es).

Specification of SRS



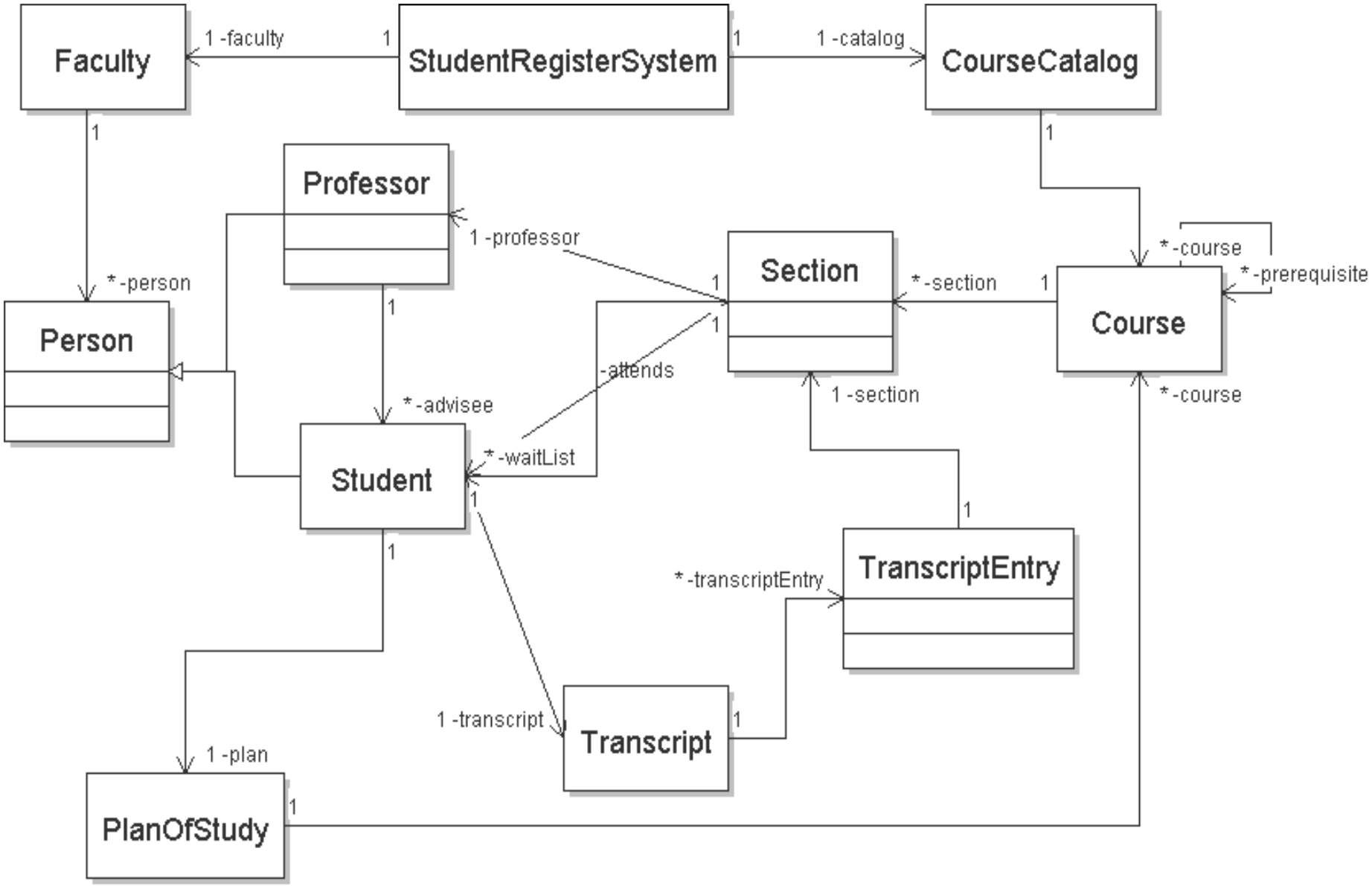
- If (a) and (b) are satisfied, but (c) is not, the student is placed on a first-come, first-served wait list. If a class/section that he/she was previously waitlisted for becomes available (either because some other student has dropped the class or because the seating capacity for the class has been increased), the student is automatically enrolled in the waitlisted class, and an email message to that effect is sent to the student. It is the student's responsibility to drop the class if it is no longer desired; otherwise, he/she will be billed for the course.
- Students may drop a class up to the end of the first week of the semester in which the class is being taught.

Classes of SRS

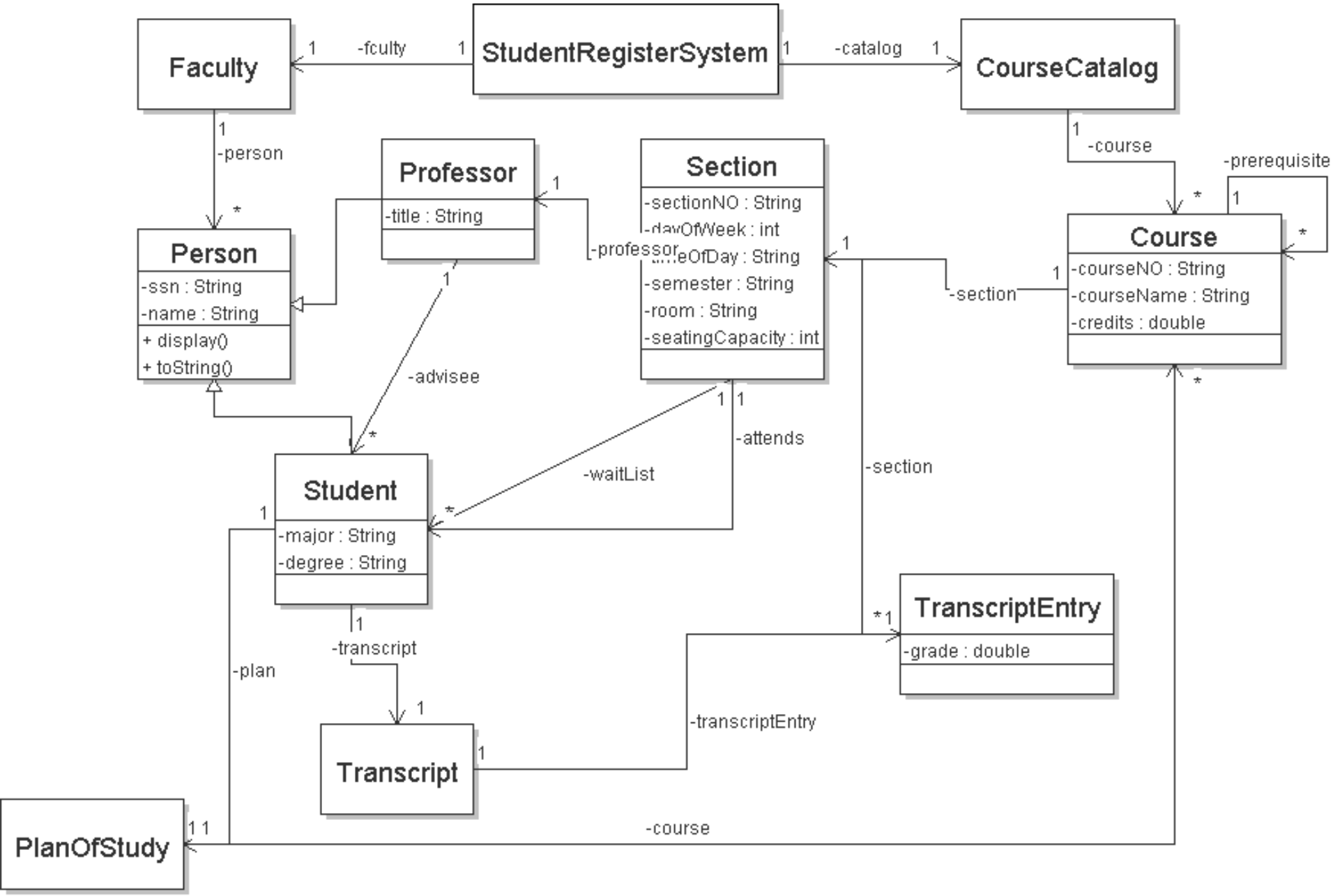


- StudentRegistrySystem
- Person
- Student
- Professor
- Course
- PlanOfStudy
- Section
- Transcript
- TranscriptEntry
- Falcuty
- CourseCatalog

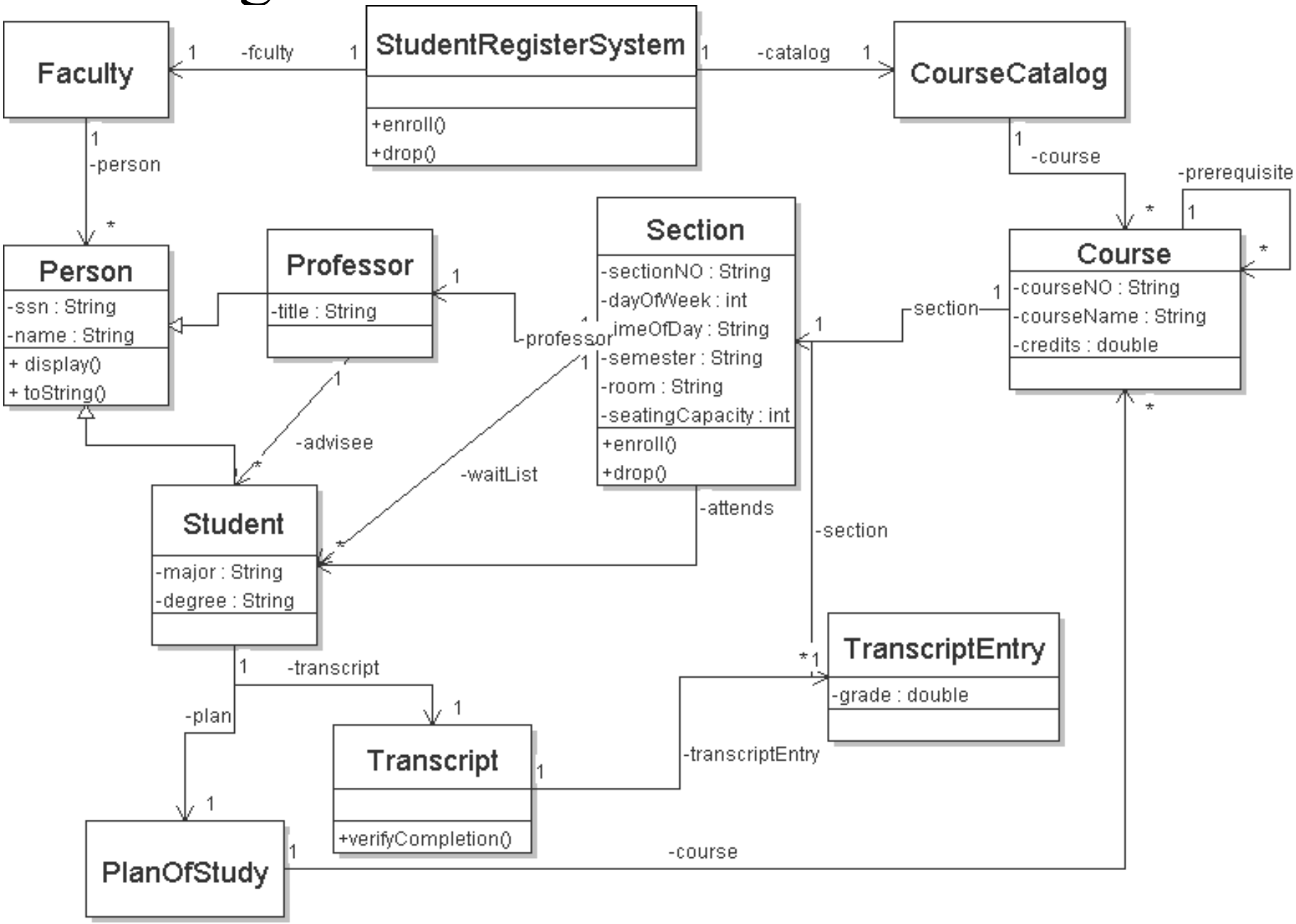
Class Relationships of SRS



Class's Attributes of SRS



Class Digram of SRS



Exercise



- Read textbook chapter 10.
- Develop a class diagram for the PTS project, using UML notation. Reflect all significant attributes and relationships among classes, including the appropriate multiplicity. Ideally, you should use an object modeling software tool if you have one available to you.